OneKhusa Go Integration Reference

Hosted Checkout & Gin Web Framework

Onekhusa Team

May 2026

1 Introduction

This document serves as a high-performance reference for developers implementing the OneKhusa Hosted Checkout flow using Go. The integration utilizes the **Gin** framework for routing and **go-socket.io** for broadcasting real-time payment confirmations to the frontend dashboard.

2 Project Directory Structure

The Go project follows a clean, modular layout to separate data models from business logic.

```
onekhusa-go-integration/
├── .env ..... Configuration & Secrets
├── go.mod ..... Module Definition
├── main.go ..... Entry Point & Webhook Controller
├── app/
│   ├── models/
│   │   └── onekhusa.go ..... Data Structs
│   └── services/
│       └── onekhusa_service.go ..... OneKhusa API Logic
└── public/
    └── index.html ..... Dashboard UI
```

3 The Brain: Environment Configuration (.env)

The `.env` file manages all external dependencies and credentials.

```
ONEKHUSA_API_KEY=sandbox_vDdLumyf...
ONEKHUSA_API_SECRET=_fefXaN9LGorR...
ONEKHUSA_ORG_ID=0BBNREQ33RSX
ONEKHUSA_MERCHANT_NUMBER=79619974

ONEKHUSA_CHECKOUT_URL=https://api.onekhusa.com/sandbox/v1/checkout/rtp/initiate
PUBLIC_CALLBACK_URL=https://zena-unjudgeable-renita.ngrok-free.dev
PORT=8080
```

Listing 1: .env Configuration

4 Data Structs (Models)

Go structs use JSON tags to map OneKhusa's camelCase API fields to Go's PascalCase naming convention.

```
type CheckoutRequest struct {
    Authentication struct {
        APIKey    string `json:"apiKey"`
        APISecret string `json:"apiSecret"`
```

```
    } `json:"authentication"`
    Payment struct {
        SourceReferenceNumber string  `json:"sourceReferenceNumber"`
        Amount                float64 `json:"amount"`
    } `json:"payment"`
    Route struct {
        SuccessRedirectionUrl string `json:"successRedirectionUrl"`
        CallbackApiUrl        string `json:"callbackApiUrl"`
    } `json:"route"`
}

type WebhookPayload struct {
    ResponseCode          string `json:"responseCode"`
    SourceReferenceNumber string `json:"sourceReferenceNumber"`
    MetaData              struct {
        ReferenceNumber string `json:"ReferenceNumber"`
    } `json:"metaData"`
}
```

Listing 2: app/models/onekhusa.go

5 Core Service Implementation

The service layer handles the HTTP client logic and applies the mandatory `X-Idempotency-Key` header.

```go
func (s *OneKhusaService) InitiateHostedCheckout(amount float64, ref string) (*models
    .CheckoutResponse, error) {
    // ... payload construction ...
    jsonPayload, _ := json.Marshal(payload)
    req, _ := http.NewRequest("POST", os.Getenv("URL"), bytes.NewBuffer(jsonPayload))

    // Set mandatory fintech headers
    req.Header.Set("X-Idempotency-Key", "GO-CHK-" + ref)
    req.Header.Set("Content-Type", "application/json")

    client := &http.Client{Timeout: 10 * time.Second}
    resp, err := client.Do(req)
    // ... decoding logic ...
}
```

Listing 3: app/services/onekhusa_service.go

6 Webhook & Real-Time Updates

The Gin controller processes the webhook and uses a Go-routine to broadcast the success event via Socket.io.

```go
r.POST("/webhooks/payments", func(c *gin.Context) {
    var payload models.WebhookPayload
    if err := c.ShouldBindJSON(&payload); err == nil {
        // Handle TitleCase 'ReferenceNumber' from Sandbox
        myRef := payload.MetaData.ReferenceNumber
        if payload.ResponseCode == "S100" {
            ticketTracker[myRef] = "Paid"
            // Broadcast to frontend
            server.BroadcastToNamespace("/", "webhook_received", gin.H{
                "reference": myRef,
                "status":    "Paid",
            })
        }
    }
    c.String(200, "acknowledged")
})
```

Listing 4: main.go Webhook Handler

7 Conclusion

Go's strict typing and efficient concurrency make it an ideal language for OneKhusa integrations. This reference ensures that developers can handle the complex redirect and webhook lifecycle while maintaining high performance and data integrity.